

Codificación de módulos del software

Brayam Manjarrez oviedo

SENA

Análisis y desarrollo de software

Ficha 3186650

Diagrama de clases

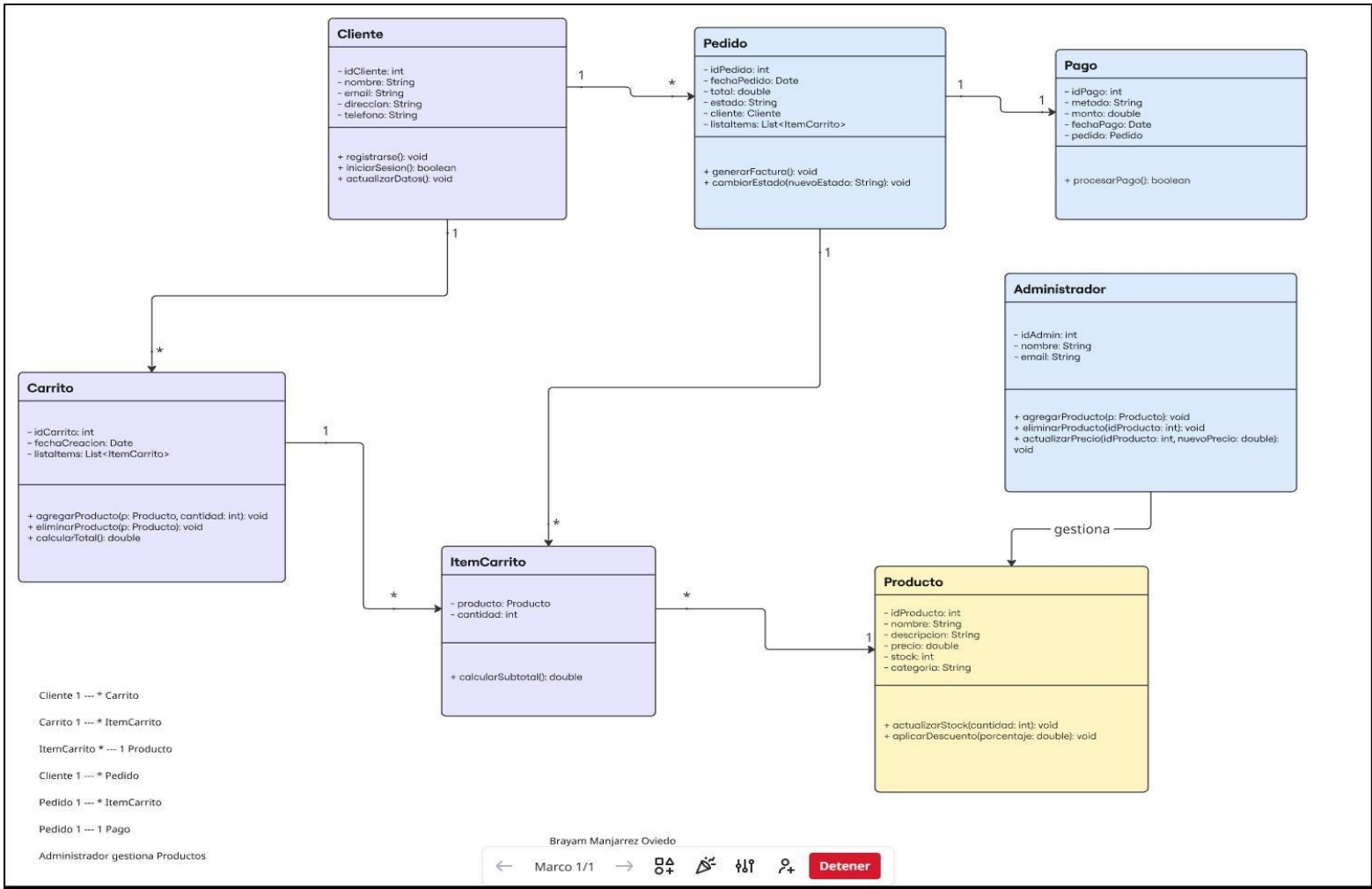


Diagrama caso de uso

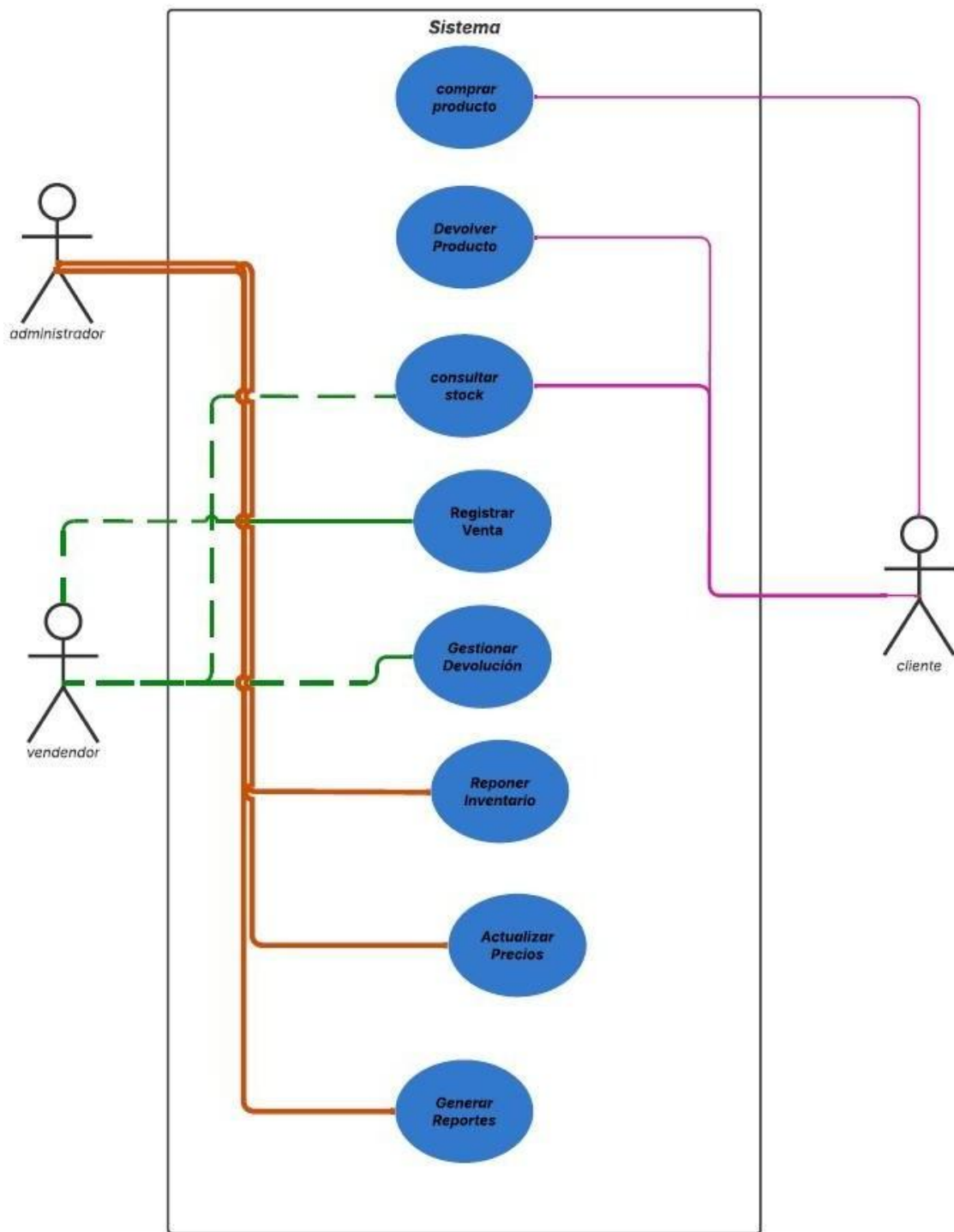


Diagrama base de datos

diagrama base datos

brayam manjerrez oviedo

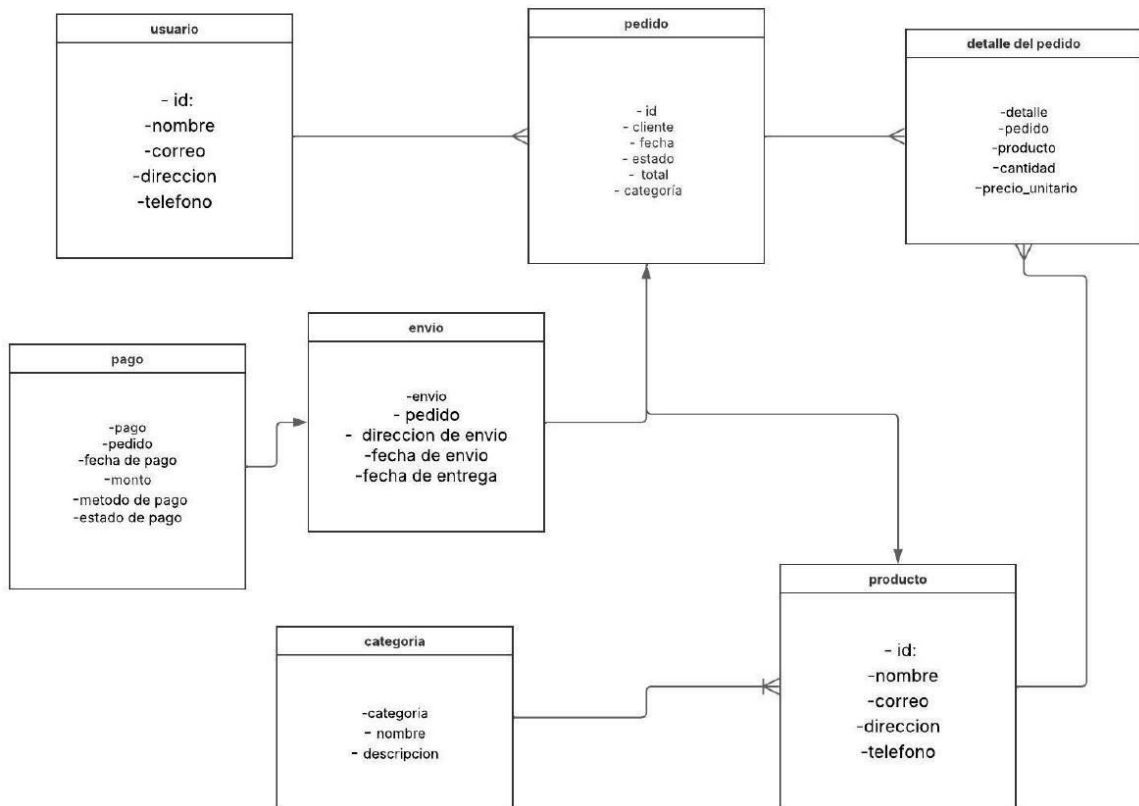
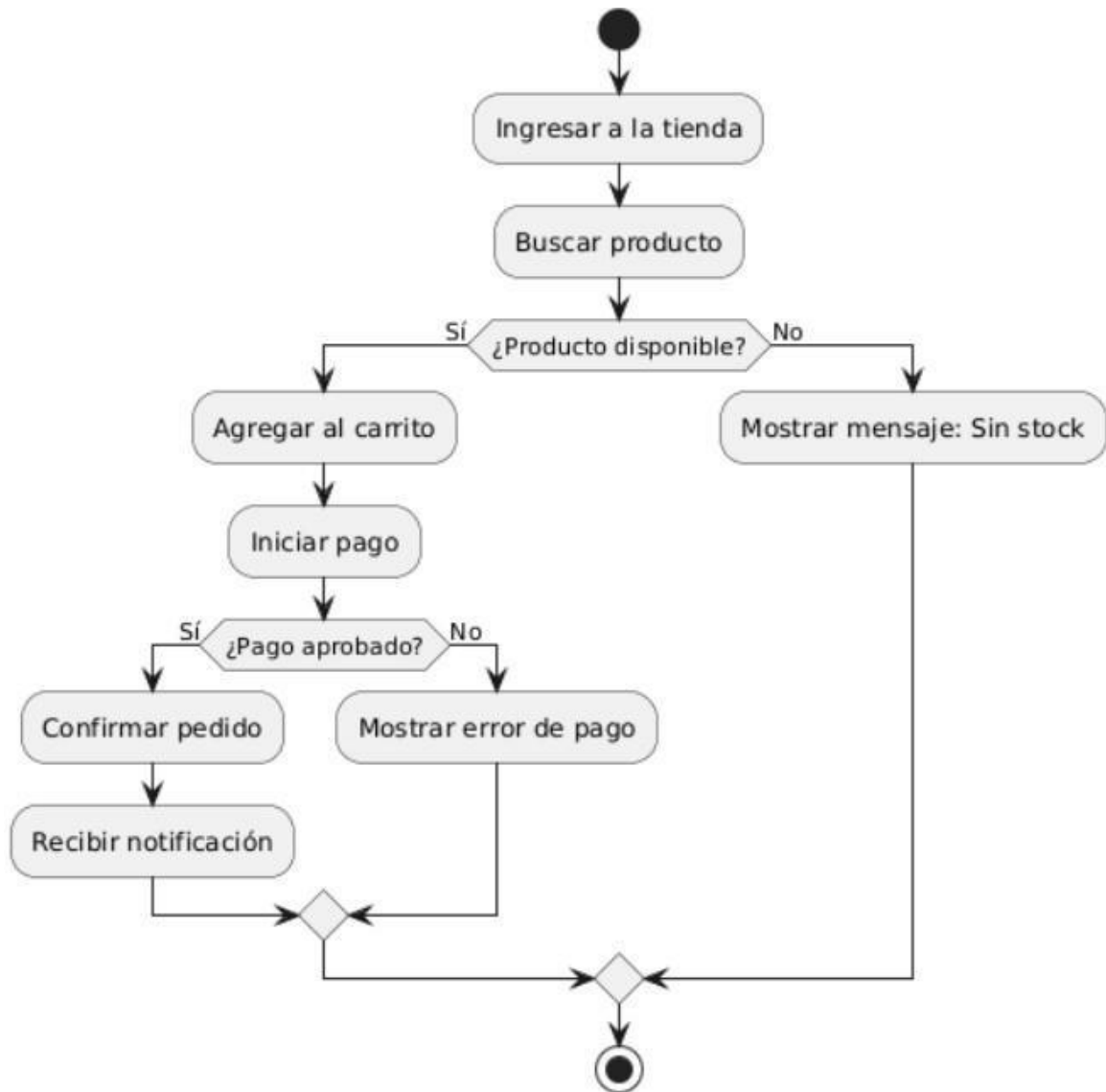


Diagrama de actividad



Aplicación de Patrones de Diseño.

Creacionales

Singleton:

Para manejar una única conexión a la base de datos en toda la aplicación, evitando duplicación de conexiones y mejorando el rendimiento. Ejemplo: Una clase ConexionBD que controla todas las operaciones con la base de datos de usuarios, pedidos y productos.

Factory Method:

Para crear diferentes tipos de productos o métodos de pago (tarjeta, PayPal, transferencia).
Ejemplo: Una clase FabricaPagos que devuelve el tipo de pago adecuado según la elección del cliente. Estructurales

Adapter:

Permite integrar servicios externos de pago o envíos (PayPal, Nequi, Servientrega) con la web.

Ejemplo: Un AdaptadorPayPalAdaptadorPayPal que traduce las solicitudes de la tienda al formato que usa PayPal.

Facade:

Crea una interfaz unificada para gestionar todo el proceso de compra. Ejemplo: Una clase SistemaVentasSistemaVentas con el método procesarCompra()procesarCompra() que coordina el carrito, el pago, el inventario y la notificación al cliente.

Comportamiento

Observer:

Para notificar automáticamente al cliente sobre el estado de su pedido (confirmado, enviado, entregado). Ejemplo: La clase Pedido notifica a los observadores (clientes) cada vez que cambia su estado.

Strategy:

Para aplicar diferentes estrategias de envío o descuentos según la región o tipo de cliente. Ejemplo: Estrategias como EnvioGratisEnvioGratis, EnvioExpressEnvioExpress, o DescuentoPorTemporada implementadas de forma intercambiable.

Prototipos interfaz usuario

Pagina de inicio

Hero Title

Our North American Field Guides provide tips for identifying birds in all of the regions of the United States and Canada. Download any of our guides for free now!

 Call To Action

Feature Message

Product Title

Attributes \$00.00

Product Title

Attributes \$00.00

Product Title

Attributes \$00.00

Product Title

Attributes \$00.00

Product Title

Attributes \$00.00

Product Title

Attributes \$00.00

Product Title

Attributes \$00.00

Product Title

Attributes \$00.00

Producto

Site Name

[Category 1](#) | [Category 2](#) | [Category 3](#)

Q

Cart (0)

[Home](#) > [Category X](#) > Product Title

Product Title

\$20

Goes Here

Size

[Sizing Chart](#)

Women

Men

Choose Size

Quantity

1

Add To Cart

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut

Related Products

Site Name

[Category X](#) | [Category Y](#) | [Category Z](#)

VISA

AM EX

Stay In Touch! Join our Newsletter.

Enter Email

Subscribe

Carrito

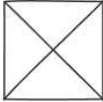
Site Name

[Category 1](#) | [Category 2](#) | [Category 3](#)

Q

 Cart (1)

Your Cart

Item	Price	Quantity
 T-Shirt Size: Small \$20.00 1 ▼		

Subtotal: \$20.00

Summary (1 item)

Subtotal

\$20.00

Shipping

-

Est. Taxes

-

Total

\$20.00

Checkout

Site Name

[Category X](#) | [Category Y](#) | [Category Z](#)



Stay In Touch! Join our Newsletter.

Enter Email

Subscribe

Envio

■ Site Name

[Category_1](#) | [Category_2](#) | [Category_3](#)

Q

 Cart (1)

Cart > Customer Information > **Shipping Information** > Payment Method

Shipping Address 12 Waldo Point Road, Mishauken, NY 11200

[Edit](#)

Shipping Method

☒ UPS Ground\$2.20

☐ UPS 3 Day Select\$5.50

☐ UPS 2nd Day Air\$9.50

☐ UPS Next Day Air\$12.50

[< Return to Customer Information](#)

[Continue to Payment Method](#)

Summary (1 item)

Subtotal\$20.00

Shipping\$2.20

Est. Taxes\$1.40

Gift card or discount code

[Apply](#)

Total\$23.60

Site Name

[Category_X](#) | [Category_Y](#) | [Category_Z](#)



Stay In Touch! Join our Newsletter.

[Subscribe](#)

Pago

Site Name

[Category 1](#) | [Category 2](#) | [Category 3](#)

Q

Cart (1)

Cart > Customer Information > Shipping Information > Payment Method

Shipping Address

12 Waldo Point Road, Mishauken, NY 11200

Edit

Shipping Method

UPS Ground, \$2.20

Edit

Payment Method

VISA

AM EX

Card number

Name on card

MM/YY

CVV

Billing Address

☒ Same as shipping address

☐ Use a different billing address

Remember Me

☐ Save my information for a faster checkout

< Return to Customer Information

Complete Order

Summary (1 item)

Subtotal

\$20.00

Shipping

\$2.20

Est. Taxes

\$1.40

Gift card or discount code

Apply

Total

\$23.60

Site Name

[Category X](#) | [Category Y](#) | [Category Z](#)

VISA

AM EX

Stay In Touch! Join our Newsletter.

Enter Email

Subscribe

Información cliente

Site Name

Category 1 | Category 2 | Category 3

Q

Cart (1)

Cart > Customer Information > Shipping Information > Payment Method

Customer Information

Already have an account? [Log In](#)

Email

Shipping Address

First Name

Last Name

Company (optional)

Address

Apt. (Optional)

Country

State

Zip

< Return to Cart

Continue to Shipping Method

Summary (1 item)

Subtotal

\$20.00

Shipping

-

Est. Taxes

-

Gift card or discount code

Apply

Total

\$20.00

Site Name

Category X | Category Y | Category Z

VISA

MasterCard

AM EX

Stay In Touch! Join our Newsletter.

Enter Email

Subscribe

Resultado de evaluación

Artefacto	Cumplimiento	Observaciones
Diagrama de casos de uso	Alto	Representa correctamente los actores y casos; todo cumple. Como mejora, se pueden añadir excepciones adicionales.
Diagrama de clases	Medio	Estructura clara y funcional; cumple, pero se puede mejorar agregando relaciones más específicas.
Diagrama de actividad	Alto	La lógica del flujo está clara y cumple; puede ampliarse con más bifurcaciones opcionales.
Modelo de base de datos	Alto	Modelo bien organizado y cumple completamente; como mejora, estandarizar nombres de tablas y campos.
Documento de diseño arquitectónico	Medio	Cumple con la descripción general; se puede mejorar agregando más detalle sobre interacción entre componentes.
Prototipo de interfaz de usuario	Alto	Interfaz clara, usable y cumple; como mejora, añadir mensajes de validación o estados.

Estructura de Nomenclatura del Proyecto

A continuación, detallo la clasificación y los nombres asignados a cada uno de los elementos (paquetes, clases y variables) utilizados en el desarrollo de este componente de software para la conexión y gestión de datos (CRUD) con MySQL.

1. Paquetes (Packages)

- **Paquete por defecto (*Default Package*):** Para este ejercicio práctico, no se definió una ruta de paquete específica en la directiva superior del archivo. La clase se encuentra ubicada directamente en la raíz del directorio de fuentes (src), lo que en Java se reconoce como el paquete por defecto.

2. Clases (Classes)

- **App:** Es la clase principal de mi proyecto. Sigue la regla de nomenclatura *UpperCamelCase* (primera letra en mayúscula). Su función es alojar el método ejecutable main y centralizar la lógica de conexión y las operaciones sobre las tablas.

3. Variables del Sistema

Las variables del código fuente se estructuraron bajo las buenas prácticas de la industria, divididas según su propósito:

A. Constantes de Conexión (Identificadores Fijos)

Definidas con los modificadores `private static final`, por lo que se nombran completamente en mayúsculas:

- **URL:** Almacena la cadena de conexión JDBC que apunta al servidor local y a mi base de datos (`jdbc:mysql://localhost:3306/base_prueba1`).
- **USUARIO:** Almacena el identificador del administrador del motor de bases de datos (`"root"`).
- **CONTRASEÑA:** Almacena la clave de seguridad para el acceso al servidor (`"brayan"`).

B. Variables de Objetos JDBC (Conectores y Canales)

Nombradas bajo la regla *lowerCamelCase* (primera palabra en minúscula, siguientes con inicial mayúscula):

- **conectar:** Instancia de la interfaz `Connection` que administra el puente de comunicación activo con MySQL.
- **statement:** Objeto de tipo `Statement` encargado de empaquetar y enviar los comandos SQL desde Java.
- **rsUsuarios:** Objeto `ResultSet` que captura el resultado de la consulta inicial a la tabla de usuarios.
- **rsProductos:** Objeto `ResultSet` que almacena los registros de la primera consulta a la tabla de productos.
- **rsFinalUsuarios:** Objeto `ResultSet` que recupera la lista de usuarios actualizada tras ejecutar la eliminación.
- **rsFinalProductos:** Objeto `ResultSet` que almacena la lista de productos modificada tras la actualización de precios.

C. Variables de Control de Datos (Lógica Local)

- **actualizarProducto:** Variable de tipo `String` que almacena la instrucción SQL (`UPDATE`) para modificar los atributos de la camiseta.
- **prodActualizados:** Variable de tipo entero (`int`) que registra la cantidad de filas afectadas en la tabla productos.
- **correoEliminar:** Variable de tipo `String` que define el parámetro de búsqueda para el borrado (`"julian@correo.com"`).
- **eliminarUsuario:** Variable de tipo `String` que almacena el comando SQL (`DELETE`) de desincorporación.
- **userEliminados:** Variable de tipo entero (`int`) que confirma cuántos registros se borraron de la tabla usuarios.

D. Variables de Extracción (Ciclo de Lectura)

Variables locales de paso corto utilizadas dentro de los ciclos while para almacenar y mostrar los datos de las columnas en la consola:

- Variables para usuarios: **id, nombre, correo.**
- Variables para productos: **id, nombre, precio, stock.**

enlace a git hub.

<https://github.com/brayamoviedo/evidencia>

Conceptos Clave y Marco Teórico.

Para lograr la comunicación entre la interfaz de usuario y la lógica de servidor, se aplicaron los siguientes conceptos fundamentales de la programación web en Java:

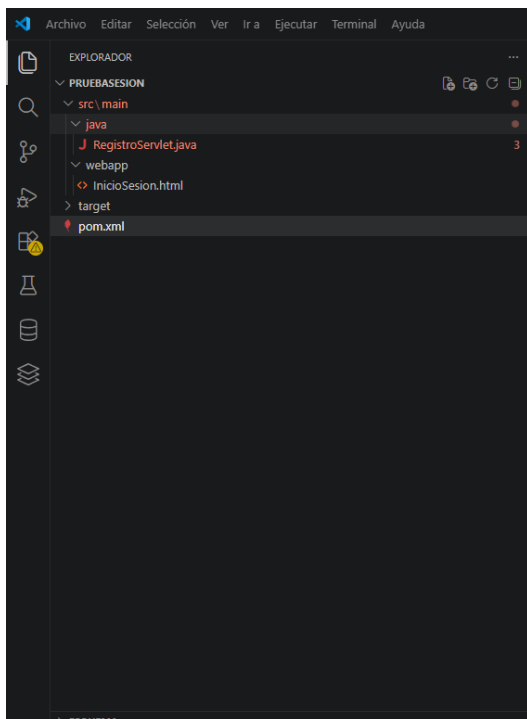
- **HTML (HyperText Markup Language):** Es el lenguaje de marcado estándar utilizado para estructurar la interfaz gráfica en el navegador. En este proyecto, provee los campos de captura (input) donde el usuario digita sus credenciales.
- **Servidor Web (Apache Tomcat):** Es un contenedor de Servlets. Actúa como el intermediario que escucha las peticiones del navegador a través de la red (puerto 8080), ejecuta el código Java correspondiente en la memoria y devuelve una respuesta estructurada.
- **Servlet (Jakarta Servlet API):** Es una clase de Java especializada en recibir peticiones dinámicas del protocolo HTTP (como formularios), procesar datos en el servidor y generar respuestas en tiempo real (páginas HTML dinámicas).
- **Maven:** Herramienta de gestión y automatización de proyectos Java. Se encarga de descargar las librerías necesarias externas (dependencias) y de empaquetar todo el código fuente en un formato distribuable.
- **Archivo .war (Web Application Archive):** Archivo comprimido estándar de Java que unifica el código compilado (.class), los archivos de configuración y las páginas web estáticas (.html) en un solo paquete listo para ser ejecutado por cualquier servidor Tomcat.

Metodología y Proceso de Desarrollo (Paso a Paso)

Para garantizar un entorno limpio y libre de errores de caché o rutas duplicadas, el proyecto se estructuró desde cero siguiendo estos pasos:

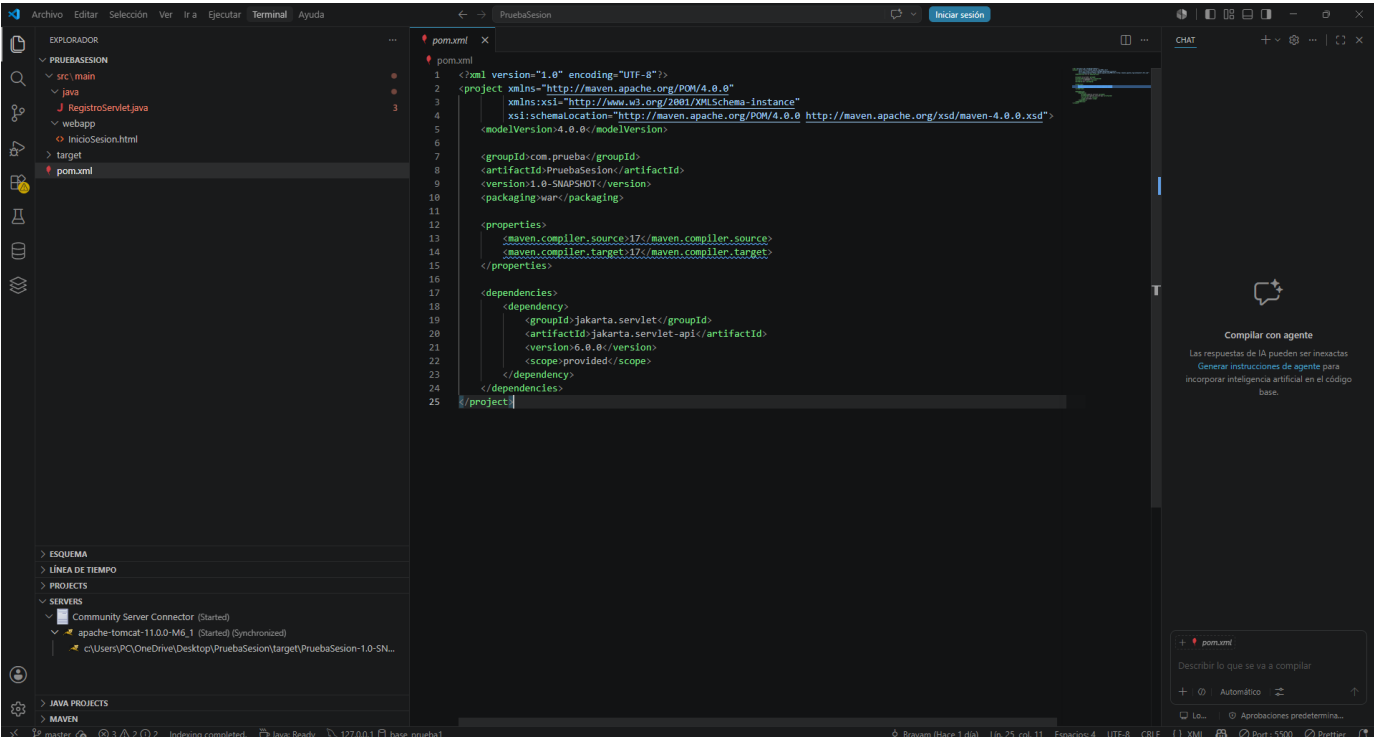
Paso 1: Configuración de la Estructura Maven

Se creó la estructura de directorios estándar exigida por Maven para aplicaciones web, asegurando la separación de componentes:



Definición de Dependencias (pom.xml)

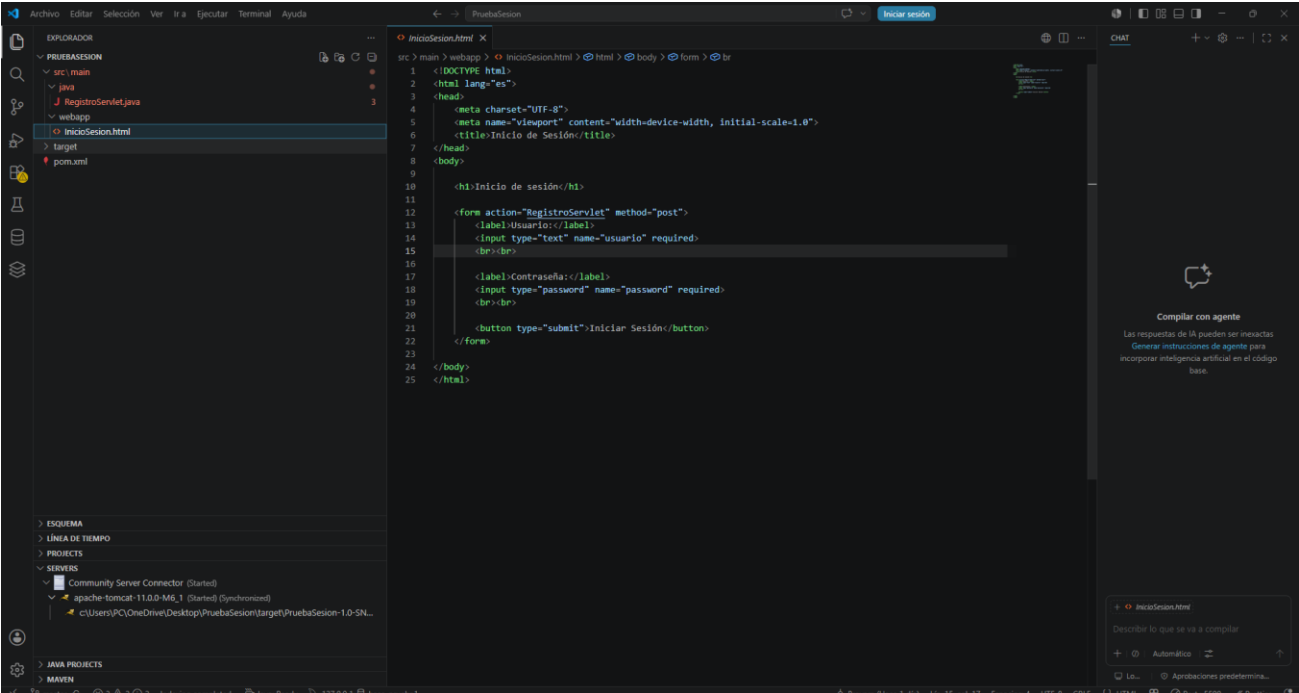
Se configuró el archivo central de Maven especificando el empaquetado tipo `<packaging>war</packaging>` y se integró la dependencia oficial de **Jakarta Servlet API (versión 6.0.0)** con el alcance `provided`, indicando que el servidor Tomcat se encargará de proveer el entorno de ejecución para dicha librería.



Diseño de la Interfaz Web (InicioSesion.html)

Se construyó un formulario web estándar. La clave técnica en este punto consistió en:

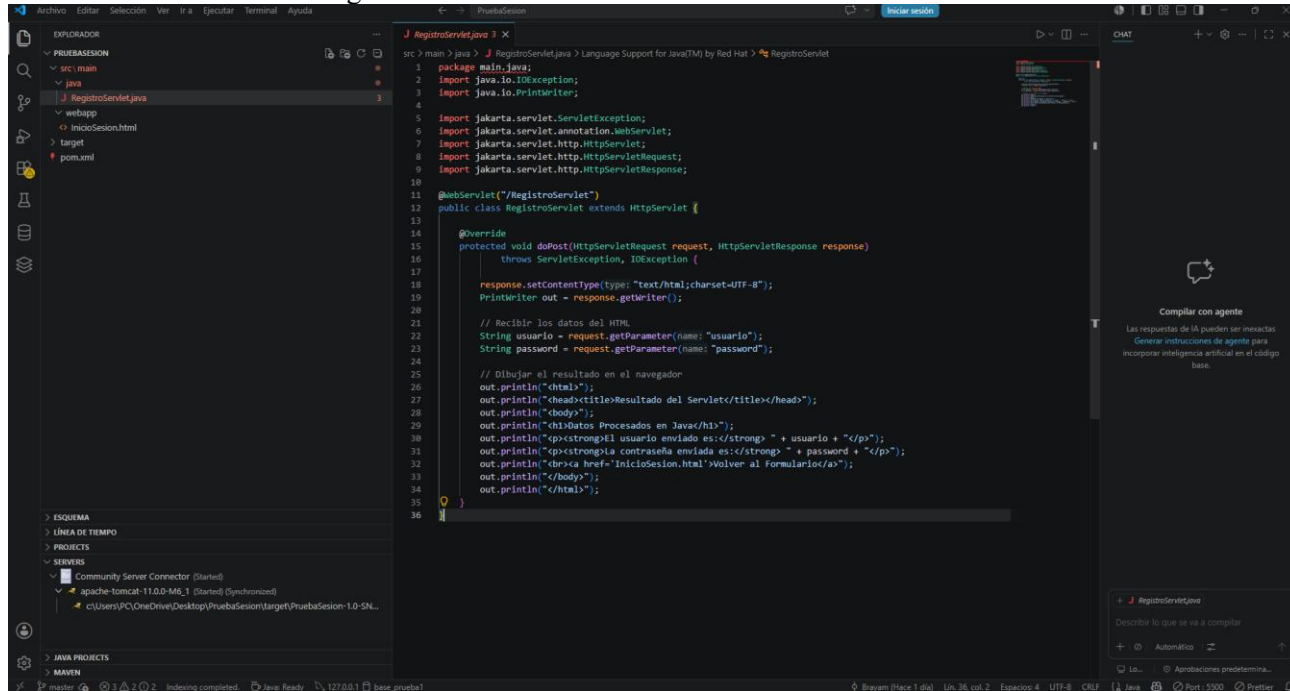
- Asignar atributos `name="usuario"` y `name="password"` a los campos de texto para que Java pueda identificarlos.
- Configurar el atributo `action="RegistroServlet"` y el método `method="post"` para enviar los datos de forma segura hacia el backend sin exponerlos en la URL.



Programación del Controlador Backend (RegistroServlet.java)

Se desarrolló la clase Java extendiendo de HttpServlet. Los hitos clave en el código fueron:

- El uso de la anotación `@WebServlet("/RegistroServlet")` para mapear la URL interna y enlazarla directamente con el formulario HTML.
- La implementación del método `doPost` para interceptar la información enviada de forma segura.
- El uso del método `request.getParameter()` para capturar y almacenar en variables de texto los datos ingresados por el usuario.
- La utilización de `PrintWriter` para renderizar y responder dinámicamente un nuevo documento HTML estructurado directamente al navegador del cliente.



Solución de Problemas Técnicos (Lecciones Aprendidas)

Durante el desarrollo se presentaron bloqueos comunes en la arquitectura cliente-servidor que fueron resueltos con criterios técnicos:

- **Error de Ejecución Local (file:///) vs. Servidor (http://):** Inicialmente, el archivo HTML se abría directamente desde el explorador de archivos o mediante servidores estáticos de desarrollo (como Live Server en el puerto 5500). Esto impedía la comunicación con el backend debido a que los navegadores locales no pueden procesar clases de Java de forma autónoma. Se corrigió forzando el acceso a través del puerto nativo de Tomcat (`http://localhost:8080/`), permitiendo que el servidor actuara como intermediario.
- **Errores de Mapeo de Rutas (404 Not Found):** Las rutas dinámicas manuales generaban inconsistencias en el servidor. Se solucionó compilando el proyecto de manera estricta mediante el comando `mvn clean package`, generando un paquete `.war` único que estandariza los accesos a los Servlets en la memoria del servidor.

Link a GIT HUB

<https://github.com/brayamoviedo/servletinicio>